



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

SEMESTRE 9

Enero - Junio 2026

Ingeniería en Sistemas Computacionales

Patrones de diseño

Reporte de práctica

Diaz Morales Katherine Giselle 22210302

Maribel Guerrero Luis

Tijuana Baja California, a 04 de marzo del 2026

Abstract Factory

El patrón Abstract Factory pertenece a la categoría de patrones creacionales y tiene como objetivo principal proporcionar una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas. Este patrón permite que un sistema sea independiente de cómo se crean, componen y representan sus objetos, promoviendo una estructura más flexible y escalable.

Abstract Factory es especialmente útil cuando se necesita trabajar con diferentes configuraciones o modalidades que comparten una estructura común. En lugar de instanciar objetos directamente utilizando la palabra reservada new, se delega la creación a una fábrica concreta que se encarga de generar los objetos correspondientes. De esta manera, el cliente solo conoce las interfaces y no las implementaciones específicas. Permite agregar nuevas familias de productos sin modificar el código existente, sino extendiéndolo mediante nuevas clases. Esto facilita el mantenimiento del sistema y reduce el riesgo de errores al momento de realizar cambios.

Código

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _1_U2_Abstract_Factory
{
    public abstract class MaterialFactory
    {
        public abstract Guia CrearGuia();
        public abstract Examen CrearExamen();
    }
    public abstract class Guia
    {
        public abstract void Mostrar();
    }
    public abstract class Examen
    {
        public abstract void Aplicar();
    }

    //presencial
    public class GuiaImpresa : Guia
    {
        public override void Mostrar()
        {
            Console.WriteLine("Mostrando la guia impresa");
        }
    }

    public class ExamenEnPapel : Examen
    {

```

```

        public override void Aplicar()
        {
            Console.WriteLine("Se aplica examen en papel");
        }
    }

    public class MaterialPresencialFactory : MaterialFactory
    {
        public override Guia CrearGuia()
        {
            return new GuiaImpresa();
        }
        public override Examen CrearExamen()
        {
            return new ExamenEnPapel();
        }
    }

    //virtual
    public class GuiaPDF : Guia
    {
        public override void Mostrar()
        {
            Console.WriteLine("Mostrando la guia PDF");
        }
    }

    public class ExamenOnline : Examen
    {
        public override void Aplicar()
        {
            Console.WriteLine("Se aplica examen en linea");
        }
    }

    public class MaterialVirtualFactory : MaterialFactory
    {
        public override Guia CrearGuia()
        {
            return new GuiaPDF();
        }

        public override Examen CrearExamen()
        {
            return new ExamenOnline();
        }
    }

    //hibrida
    public class GuiaHibrida : Guia
    {
        public override void Mostrar()
        {
            Console.WriteLine("Mostrando la guia en modalidad hibrida");
        }
    }

    public class ExamenMixto : Examen
    {
        public override void Aplicar()
        {

```

```

        Console.WriteLine("Se aplica examen mixto");
    }
}
public class MaterialHibridaFactory : MaterialFactory
{
    public override Guia CrearGuia()
    {
        return new GuiaHibrida();
    }

    public override Examen CrearExamen()
    {
        return new ExamenMixto();
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        MaterialFactory fabrica;
        fabrica = new MaterialPresencialFactory();
        Guia guia = fabrica.CrearGuia();
        Examen examen = fabrica.CrearExamen();
        guia.Mostrar();
        examen.Aplicar();
        Console.WriteLine("");

        fabrica = new MaterialVirtualFactory();
        guia = fabrica.CrearGuia();
        examen = fabrica.CrearExamen();
        guia.Mostrar();
        examen.Aplicar();
        Console.WriteLine("");

        fabrica = new MaterialHibridaFactory();
        guia = fabrica.CrearGuia();
        examen = fabrica.CrearExamen();
        guia.Mostrar();
        examen.Aplicar();
        Console.ReadKey();
    }
}

```

Conclusión

Abstract factory ayuda a mantener el código estructurado y organizado, lo cual es importante en el desarrollo de software, especialmente cuando el sistema puede crecer o cambiar con el tiempo, por ejemplo, al agregar la modalidad híbrida se pudo observar que fue posible extender el sistema sin modificar el código, lo que demuestra la flexibilidad que ofrece este patrón.